

# Machine Learning

Week 5 Lecture

Dr. Muhammad Ahmed Raza

# Contents

- Intro to Decision Trees
- Decision Trees
  - Feature Selection
  - Overfitting
- Pruning Strategies

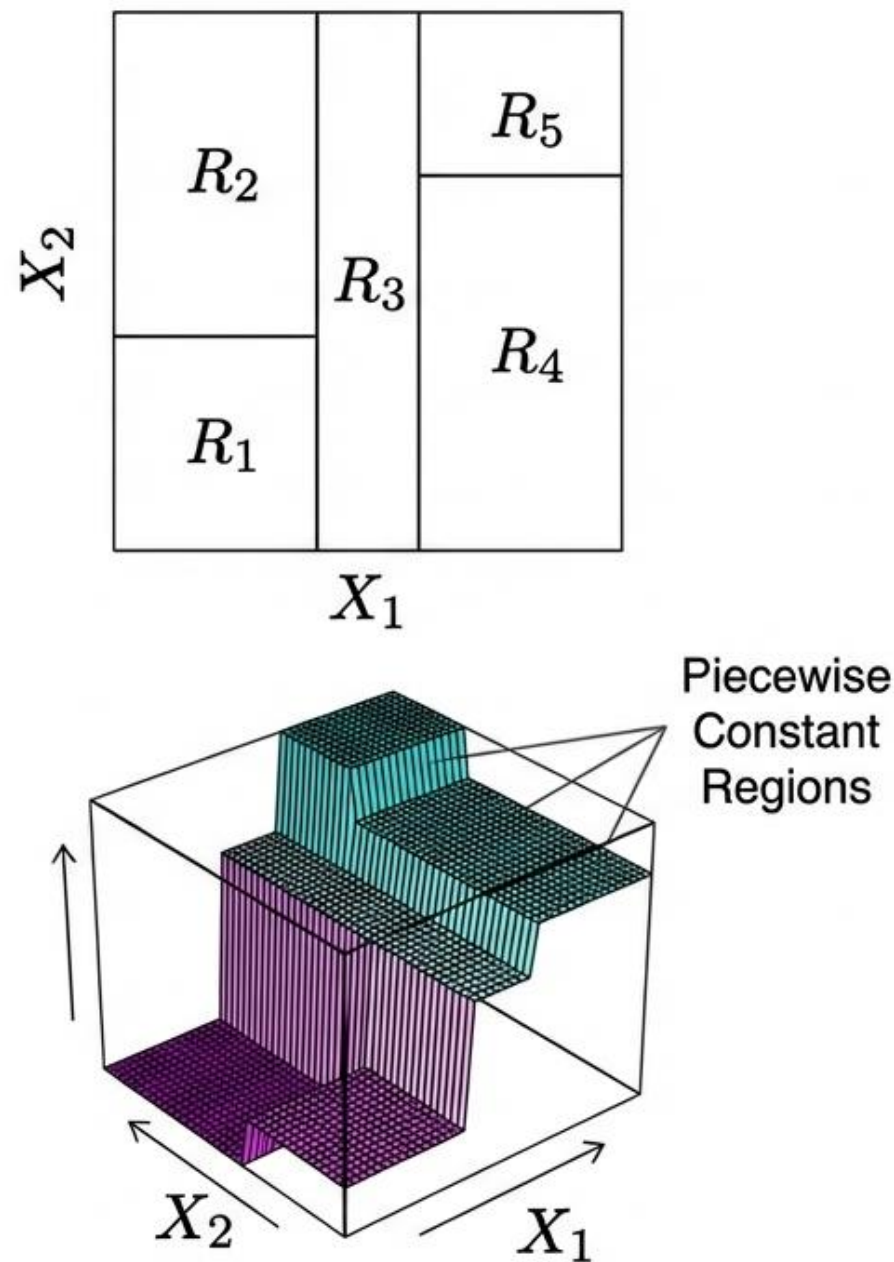
# From Continuous to Discrete Optimization

- So far:
  - Gradient Descent → minimize smooth function
  - Ridge / Lasso
- Today:
  - No gradients
  - No convexity
  - We optimize a combinatorial structure
- Decision Trees:
  - Minimize Impurity

# Recursive Partitioning

Decision trees approximate complex functions by **slicing data space** into manageable, **piecewise constant regions**. The algorithm asks a series of **binary questions** to isolate **homogeneous groups**.

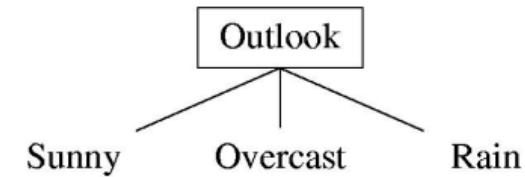
- The Strategy: Divide and Conquer.



# Decision Tree (Categorical features)

$\langle x_i, y_i \rangle$

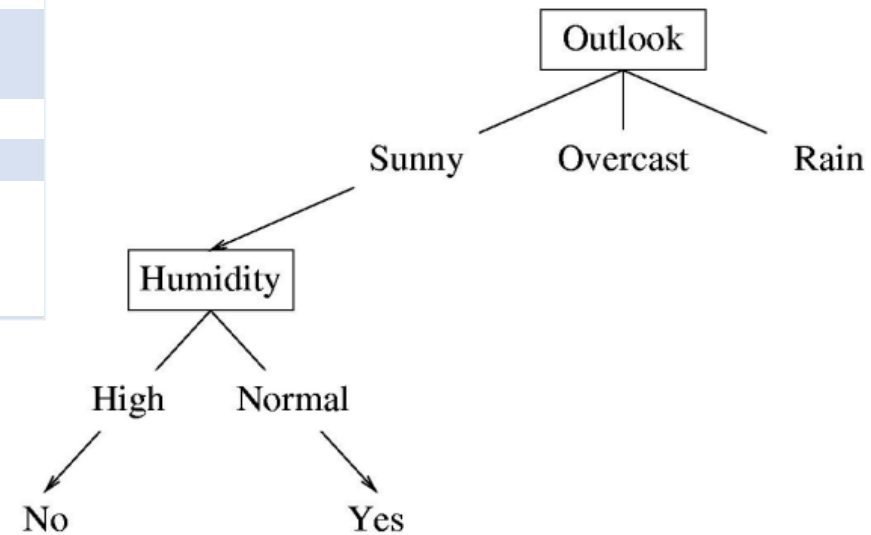
| Predictors |             |          |        | Response |
|------------|-------------|----------|--------|----------|
| Outlook    | Temperature | Humidity | Wind   | Class    |
| Sunny      | Hot         | High     | Weak   | No       |
| Sunny      | Hot         | High     | Strong | No       |
| Overcast   | Hot         | High     | Weak   | Yes      |
| Rain       | Mild        | High     | Weak   | Yes      |
| Rain       | Cool        | Normal   | Weak   | Yes      |
| Rain       | Cool        | Normal   | Strong | No       |
| Overcast   | Cool        | Normal   | Strong | Yes      |
| Sunny      | Mild        | High     | Weak   | No       |
| Sunny      | Cool        | Normal   | Weak   | Yes      |
| Rain       | Mild        | Normal   | Weak   | Yes      |
| Sunny      | Mild        | Normal   | Strong | Yes      |
| Overcast   | Mild        | High     | Strong | Yes      |
| Overcast   | Hot         | Normal   | Weak   | Yes      |
| Rain       | Mild        | High     | Strong | No       |



# Decision Tree (Categorical features)

| Predictors |             |          |        | Response |
|------------|-------------|----------|--------|----------|
| Outlook    | Temperature | Humidity | Wind   | Class    |
| Sunny      | Hot         | High     | Weak   | No       |
| Sunny      | Hot         | High     | Strong | No       |
|            |             |          |        |          |
| Sunny      | Mild        | High     | Weak   | No       |
| Sunny      | Cool        | Normal   | Weak   | Yes      |
|            |             |          |        |          |
| Sunny      | Mild        | Normal   | Strong | Yes      |

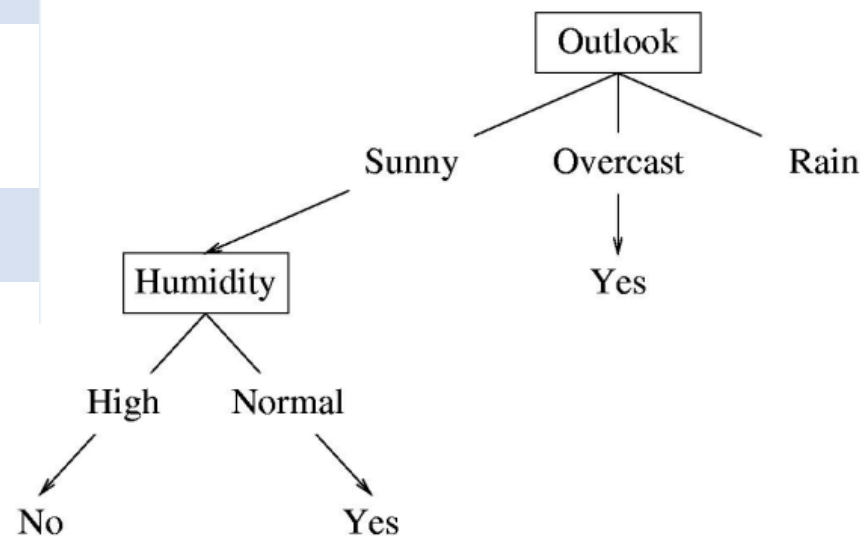
$\langle \mathbf{x}_i, y_i \rangle$



# Decision Tree (Categorical features)

$\langle \mathbf{x}_i, y_i \rangle$

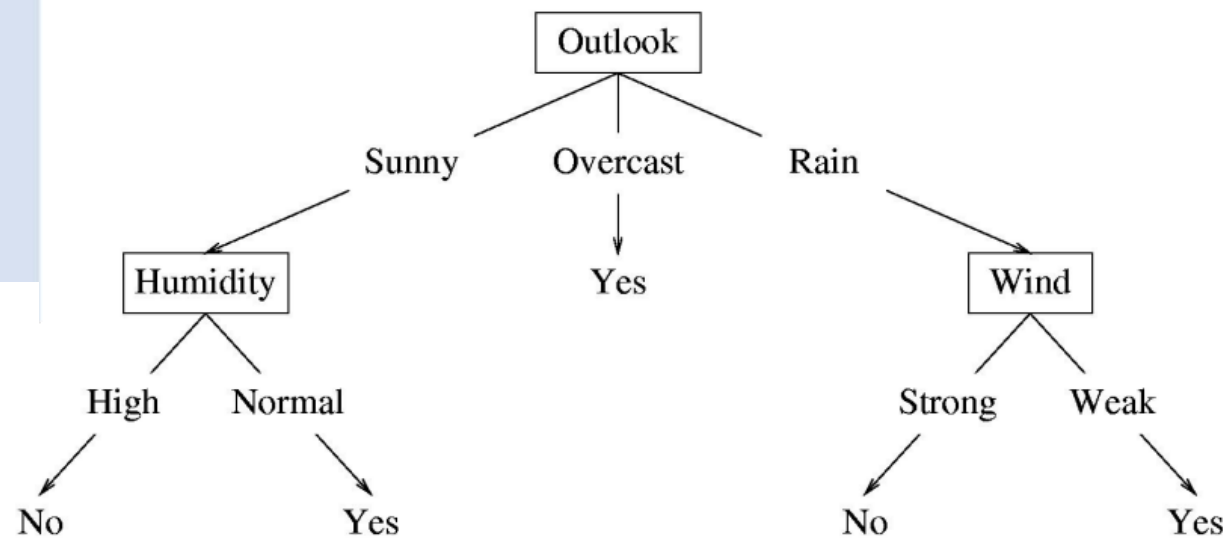
| Predictors |             |          |        | Response |
|------------|-------------|----------|--------|----------|
| Outlook    | Temperature | Humidity | Wind   | Class    |
| Overcast   | Hot         | High     | Weak   | Yes      |
| Overcast   | Cool        | Normal   | Strong | Yes      |
| Overcast   | Mild        | High     | Strong | Yes      |
| Overcast   | Hot         | Normal   | Weak   | Yes      |



# Decision Tree (Categorical features)

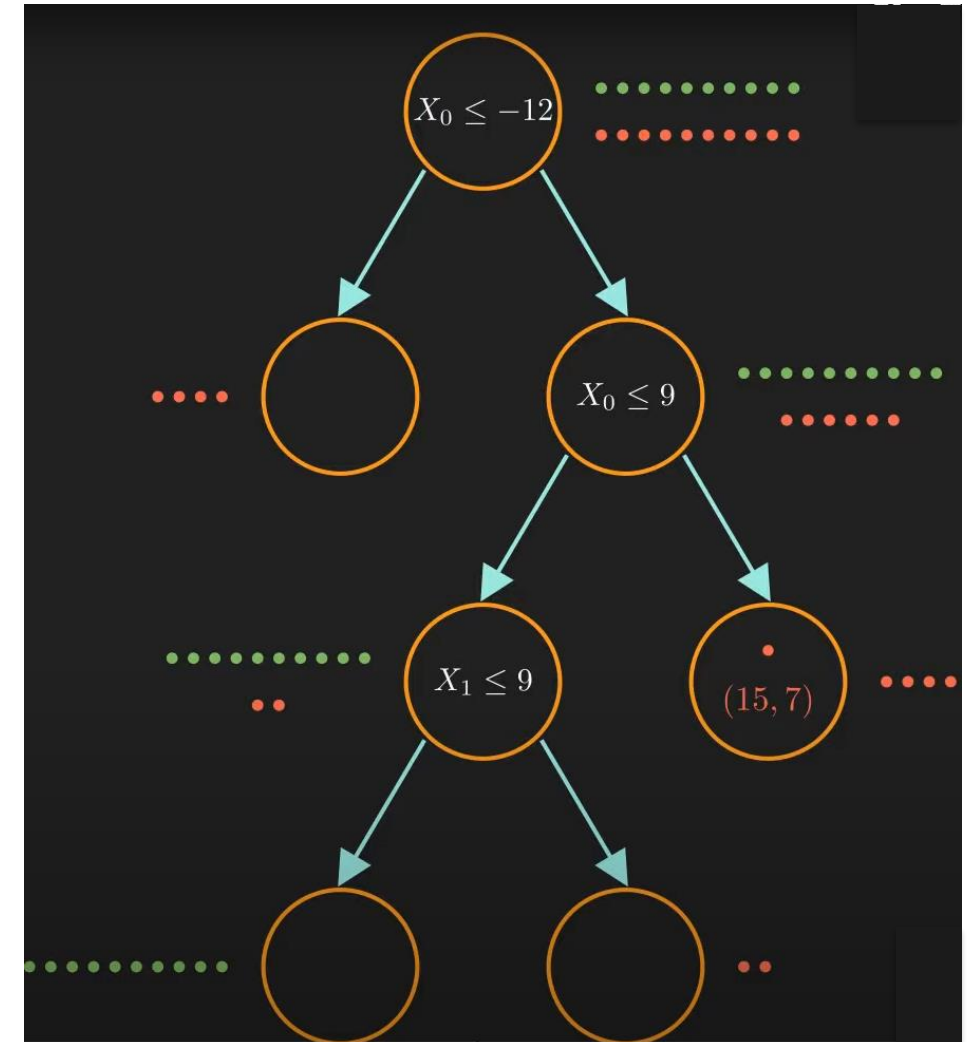
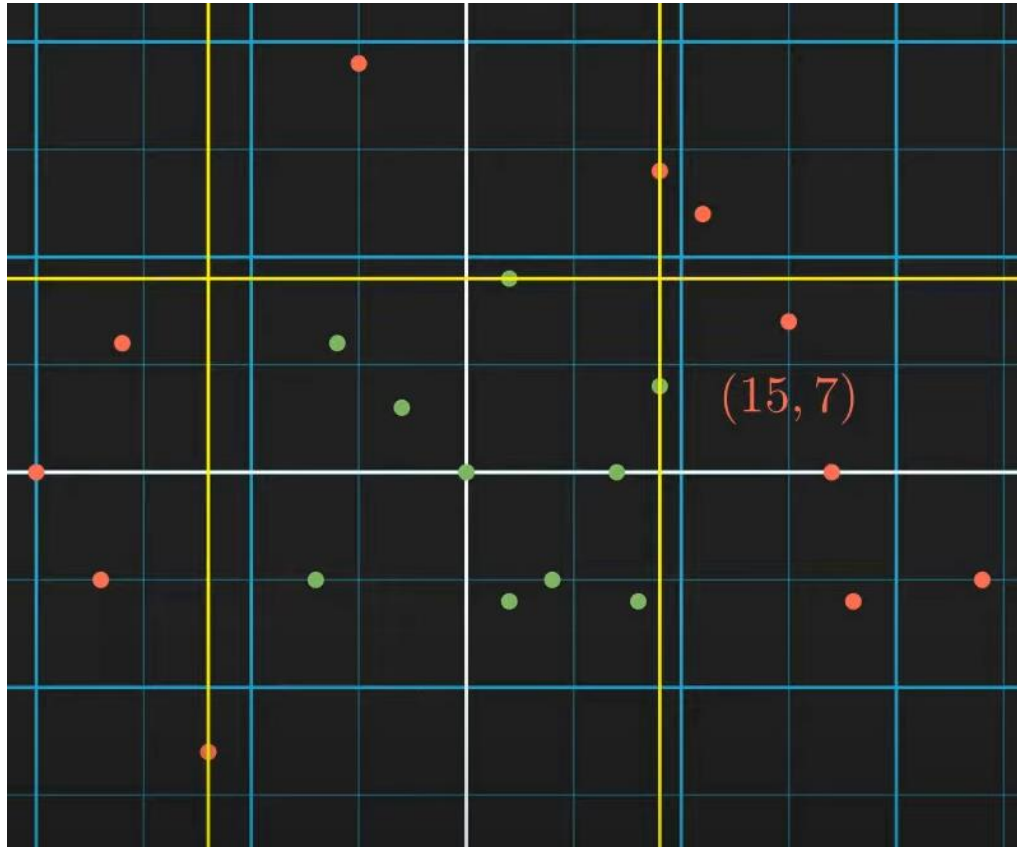
$\langle \mathbf{x}_i, y_i \rangle$

| Predictors |             |          |        | Response |
|------------|-------------|----------|--------|----------|
| Outlook    | Temperature | Humidity | Wind   | Class    |
| Sunny      | Hot         | High     | Weak   | No       |
| Sunny      | Hot         | High     | Strong | No       |
| Overcast   | Hot         | High     | Weak   | Yes      |
| Rain       | Mild        | High     | Weak   | Yes      |
| Rain       | Cool        | Normal   | Weak   | Yes      |
| Rain       | Cool        | Normal   | Strong | No       |
| Overcast   | Cool        | Normal   | Strong | Yes      |
| Sunny      | Mild        | High     | Weak   | No       |
| Sunny      | Cool        | Normal   | Weak   | Yes      |
| Rain       | Mild        | Normal   | Weak   | Yes      |
| Sunny      | Mild        | Normal   | Strong | Yes      |
| Overcast   | Mild        | High     | Strong | Yes      |
| Overcast   | Hot         | Normal   | Weak   | Yes      |



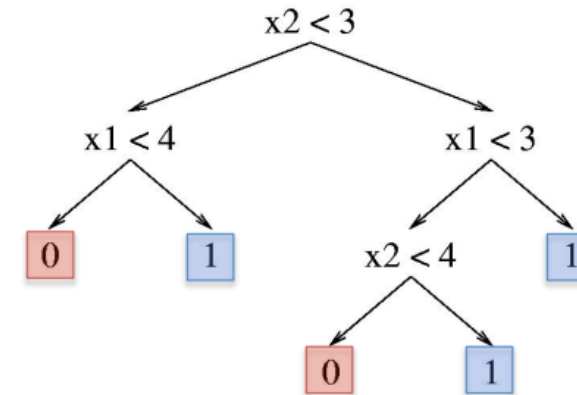
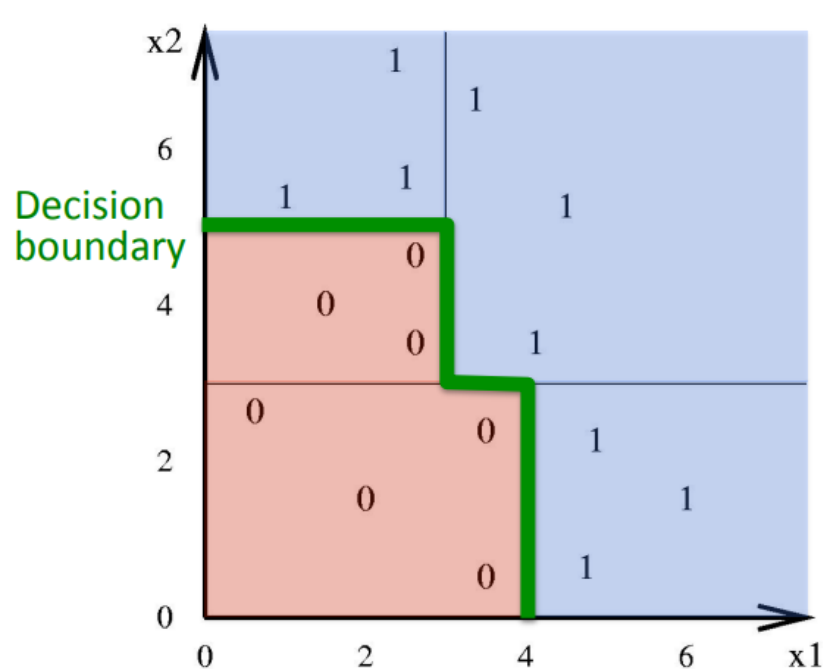


# Decision Tree (Continuous features)



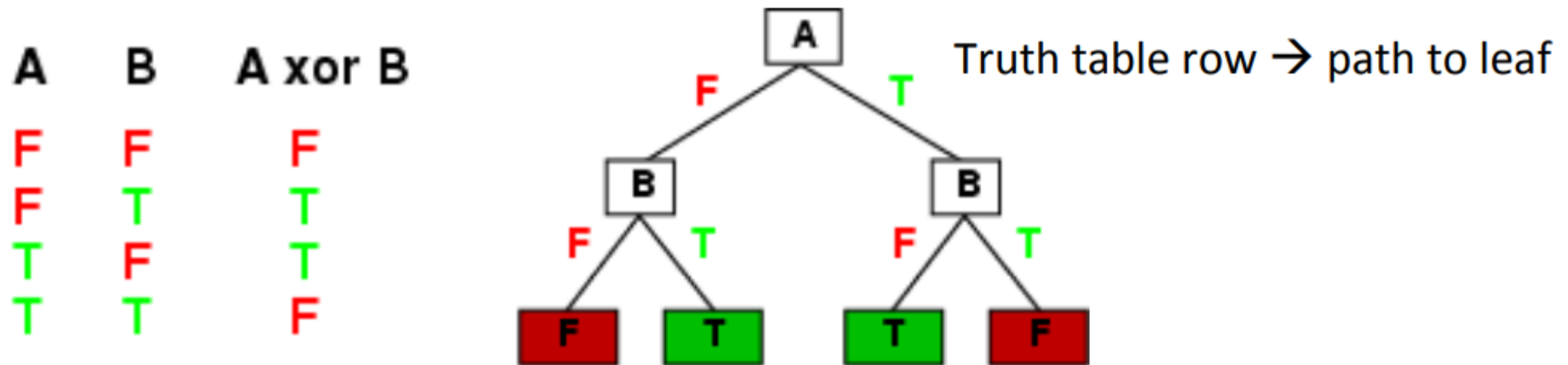
# Decision Tree – Decision Boundary

- Decision trees divide the feature space into axis parallel (hyper-)rectangles
- Each rectangular region is labelled with one label or a probability distribution over labels



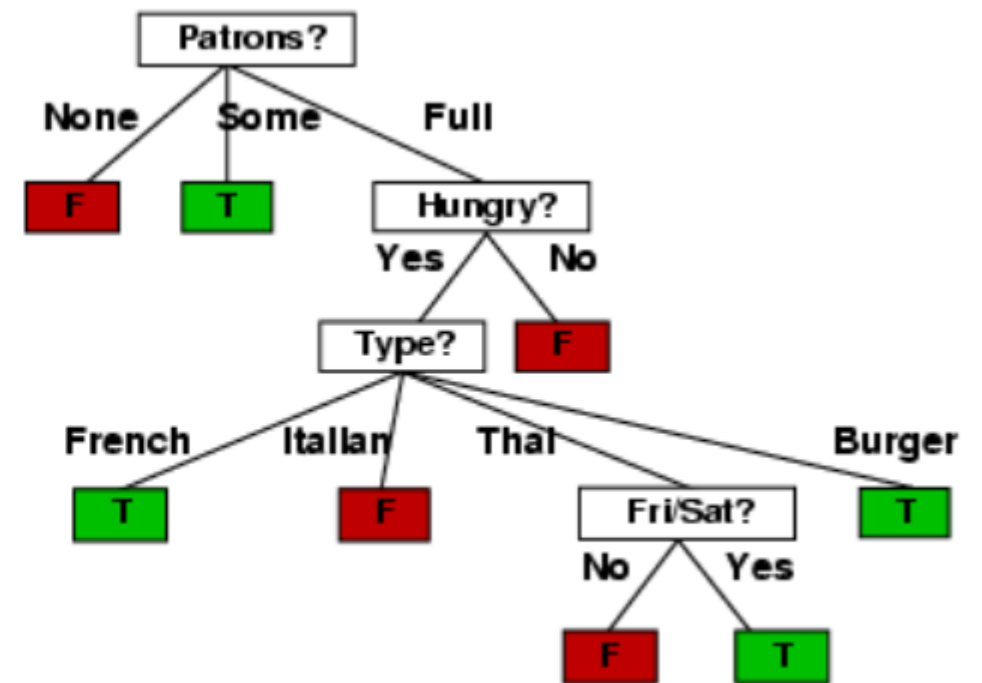
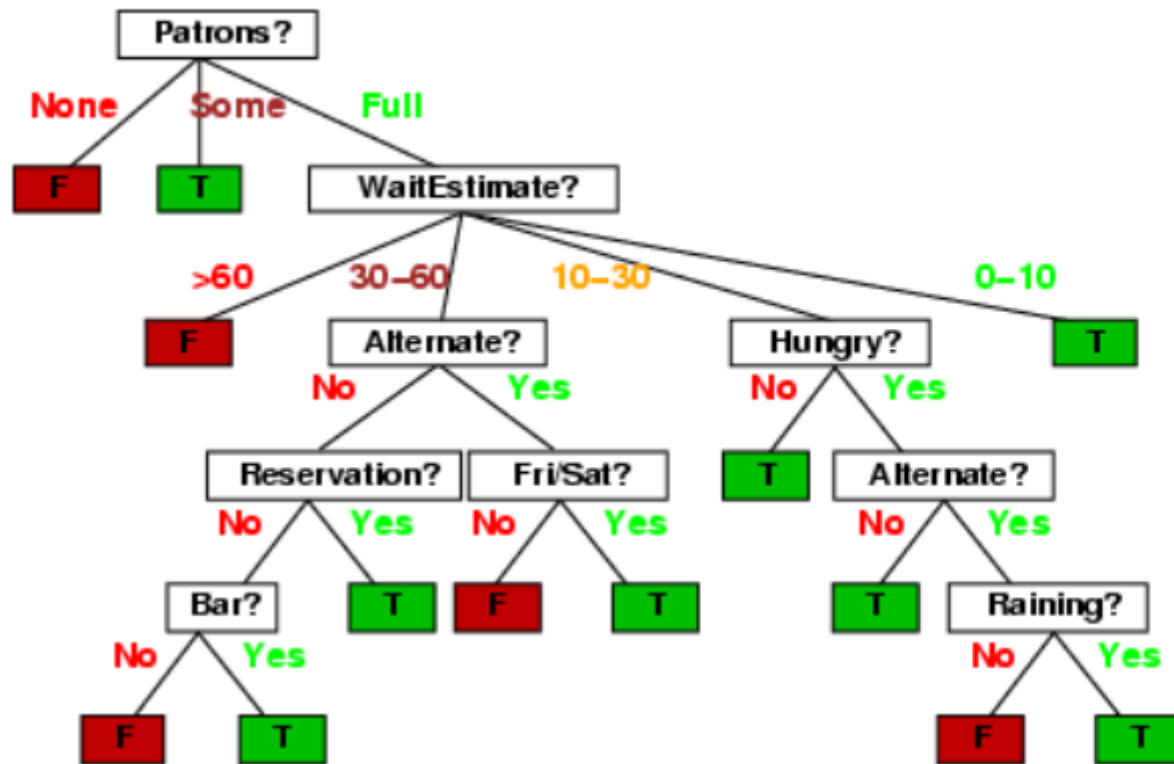
# Expressiveness

- Decision trees can represent any boolean function of the input features



- In the worst case, the tree will require exponentially many nodes

# Choosing the Best Features



# Choosing the Best Features

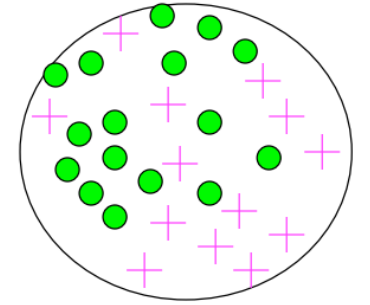
- Key Problem: Choosing the Best Features/attributes.
- Some possibilities are:
  - Random: Select any features at random
  - Least-Values: Choose the features with the smallest number of possible values
    - e.g., binary features (only two possibilities)
  - Most-Values: Choose the features with the largest number of possible values
  - Max-Gain: Choose the features that has the largest expected information gain
- The ID3 algorithm uses the Max-Gain method of selecting the best features

# Entropy Definition

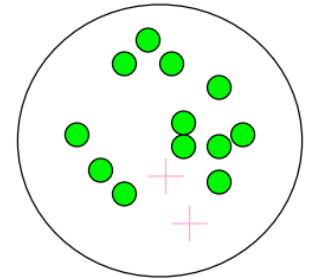
$$H(p) = - \sum_{i=1}^N p_i \log(p_i)$$

- Measures the level of impurity in a group of examples
- Properties:
  - $H = 0$ , if pure
  - Maximum when uniform
  - Strictly concave
- Interpretation:
  - Expected information (uncertainty).

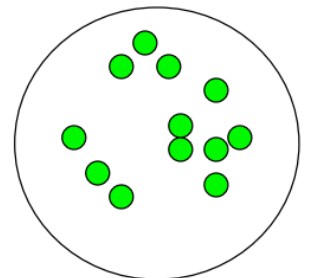
**Very impure group**



**Less impure**

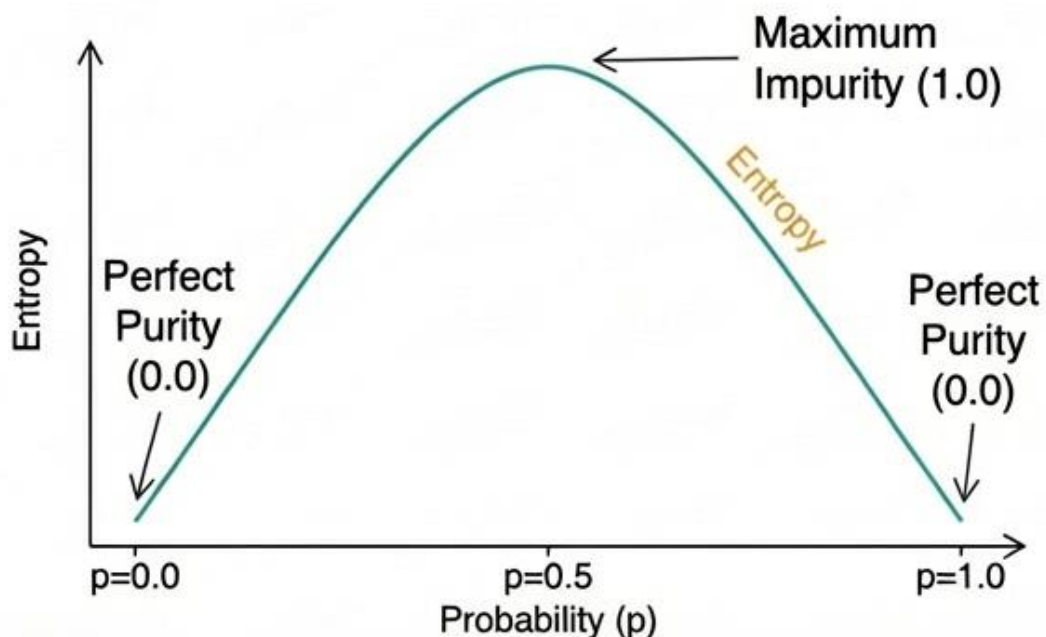


**Minimum  
impurity**



# The Metric of Uncertainty

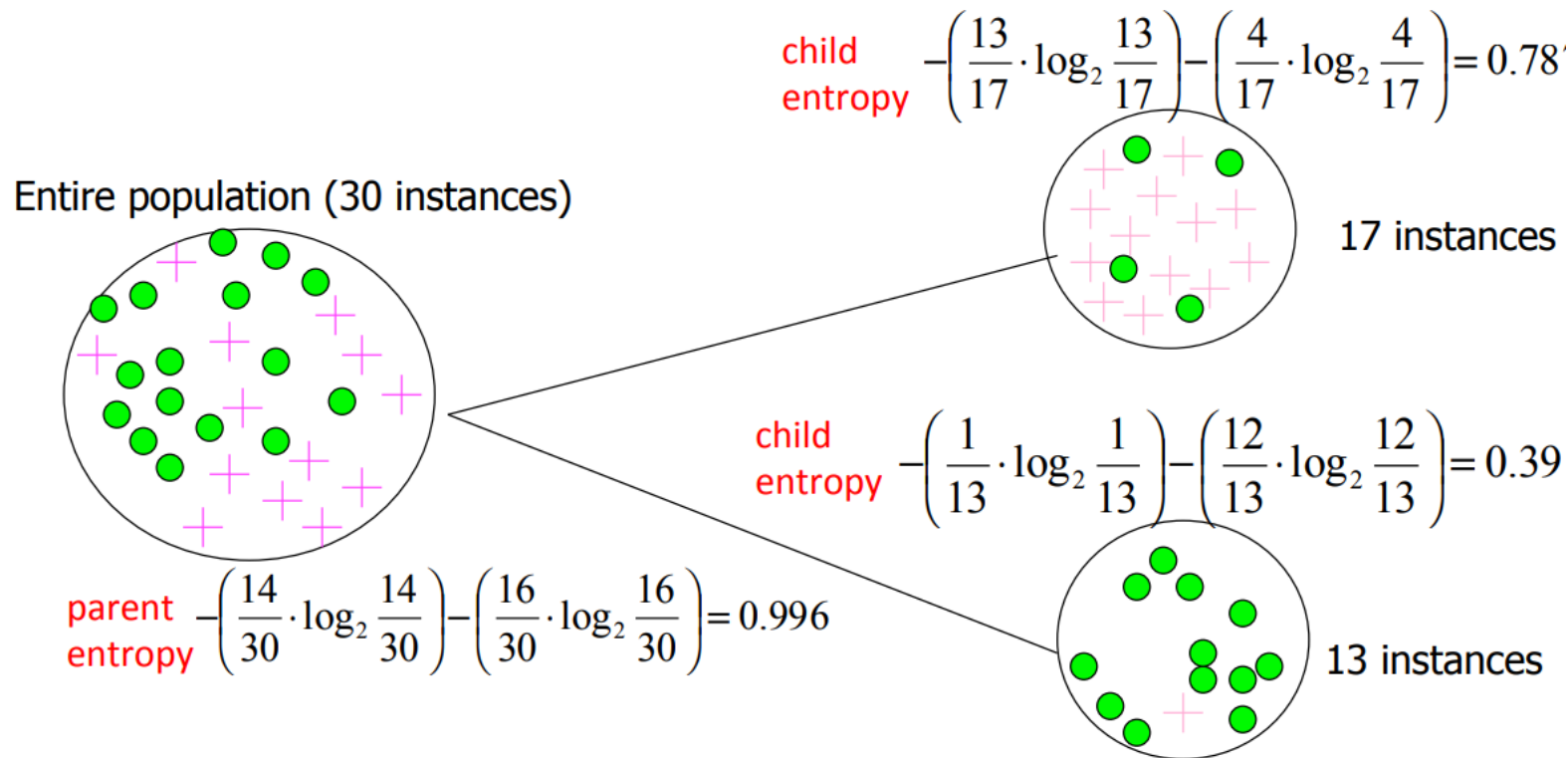
How does the tree decide where to split? It seeks to minimize **Entropy (impurity)**. The goal is to maximize **Information Gain**—the reduction in chaos achieved by a specific split.



$$\text{Entropy } (P) = - \sum_{i=1}^n \overset{\substack{\text{Probability of the class} \\ \text{(e.g., proportion of 'Yes')}}}{p_i} \times \log(p_i)$$

# Information Gain

**Information Gain** = entropy(parent) – [average entropy(children)]



(Weighted) Average Entropy of Children =  $\left(\frac{17}{30} \cdot 0.787\right) + \left(\frac{13}{30} \cdot 0.391\right) = 0.615$

**Information Gain = 0.996 - 0.615 = 0.38**



# Decision Tree

- Algorithm:
  - Suppose we have a dataset  $D$
  - Define a split for a feature  $x_n < t$ , where  $t$  is the threshold.
    - We divide the data  $D$  into  $\{D_{left}, D_{right}\}$

- Compute Impurity (Gini or Entropy)

- Gini Impurity 
$$G(\mathcal{D}) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

$K$  is number of classes

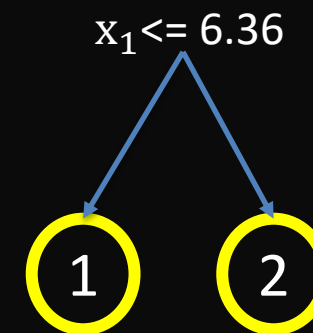
- Entropy 
$$H(\mathcal{D}) = - \sum_{k=1}^K p_k \log_2(p_k)$$

$n$  total number of samples  
 $n_l$  total samples in left split  
 $n_r$  total samples in the right split

- Compute Information Gain

$$IG = H(D) - \left( \frac{n_L}{n} H(D_{left}) + \frac{n_R}{n} H(D_{right}) \right)$$

# Demo Video



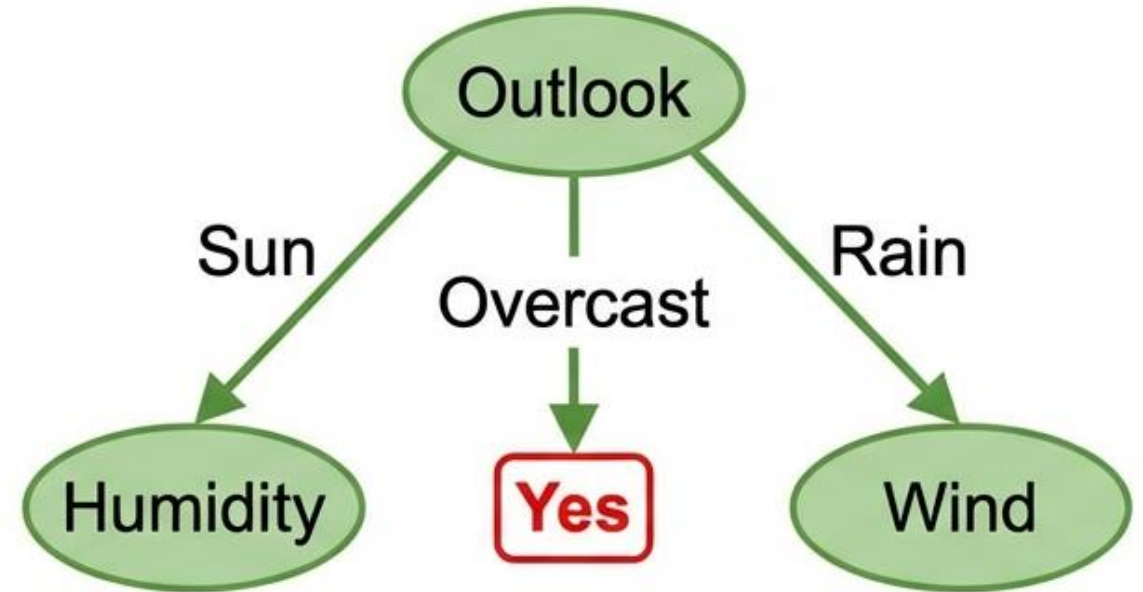
# Early Stopping Criteria

- Stop splitting when:
  - max\_depth reached
  - min\_samples\_split
  - min\_samples\_leaf
  - $IG < \text{threshold}$
- Interpretation:
  - Hard constraints on hypothesis space.

# ID3: The Greedy Builder

Developed by J. Ross Quinlan, ID3 (Iterative Dichotomiser 3) builds the tree top-down. It iterates through unused attributes, calculates Entropy, and selects the attribute that maximizes Information Gain *right now*.

Constraint: Categorical data only. No missing values.

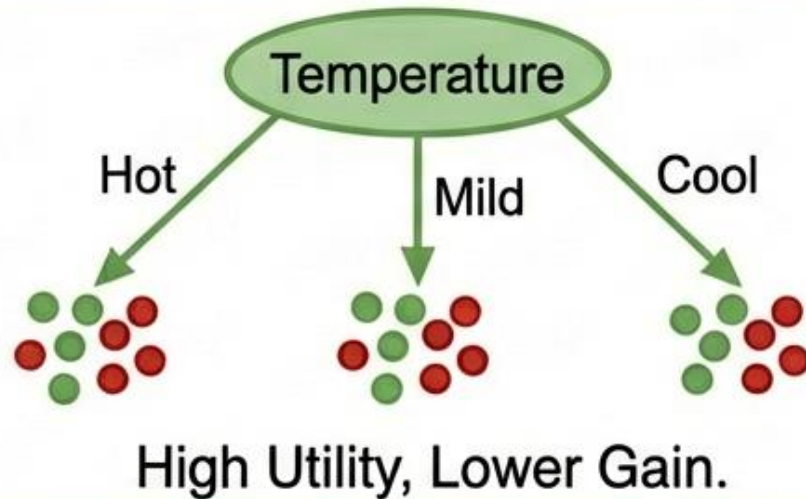


ID3 splits the dataset  $S$  based on the attribute yielding the highest immediate gain (Outlook = 0.246).

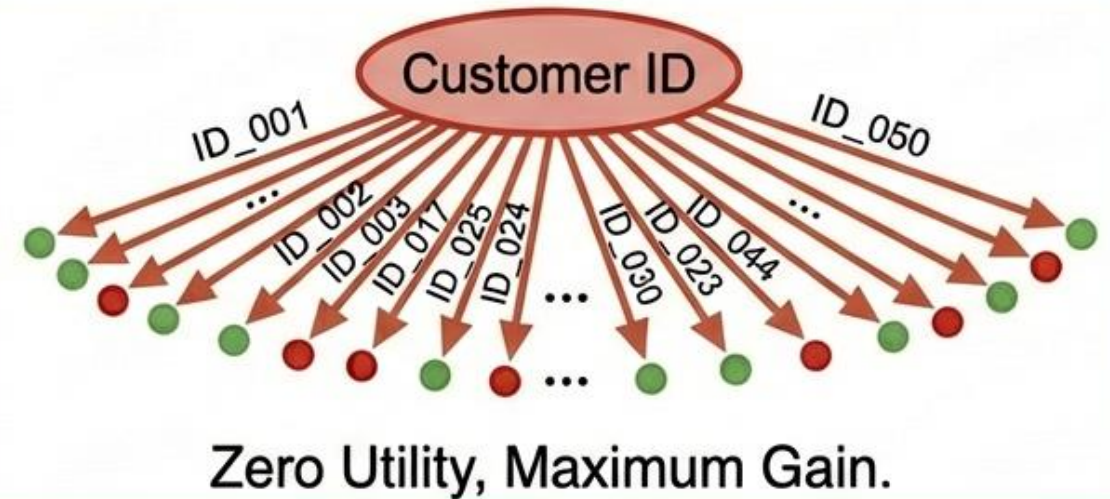
# The Flaw in the Design: Cardinality Bias

ID3 is biased toward attributes with many unique values. A unique identifier (like a Social Security Number) yields perfect purity (Zero Entropy) but has zero predictive value for new data.

**Do This:** High Utility, Lower Gain



**Not That:** Zero Utility, Maximum Gain



ID3 foolishly chases the granular split, memorizing the training data instead of learning from it.



# Enter C4.5: The Adapted Species

To fix the cardinality bias, C4.5 introduces “Gain Ratio”. It penalizes attributes that fragment the data into too many tiny pieces.

$$\text{GainRatio}(p,T) = \frac{\text{Gain}(p,T)}{\text{SplitInfo}(p,T)}$$

Split Info: Measures the "intrinsic information" of the feature. If a feature has many possible values, the Split Information value will be high, which **shrinks** the final Gain Ratio.

**The Penalty Term:** Punishes splits that create broad, shallow fragmentation.

# Pruning Architectures

Pruning removes dead branches to reveal the healthy structure.



**Pre-Pruning:** Stops growth early. Fast, but risks missing patterns (Horizon Effect).



**Post-Pruning:** Grows full, then cuts back. Slower, but more accurate.

# Pre vs Post Pruning

- Pre-pruning (early stopping) halts tree growth using constraints like max depth, min samples per split/leaf or statistical tests — it's fast and memory-efficient but can underfit by cutting useful splits.
- Post-pruning grows a full tree then removes branches using validation-based methods (reduced-error, cost-complexity, cross-validation) — usually yields better generalization but is slower and needs extra data/compute.



# Pre-Pruning

- Is basically early stopping:
  - Conceptually:
    - "Stop learning now, because if you keep going, you're just going to start memorizing the noise in the background."
  - Risks:
    - **Pre-pruning** is "short-sighted." It might stop a split because the immediate improvement is low

# Post Pruning

- Post-pruning grows a full tree then removes branches
- Risks
  - **Post-pruning** is "hindsight." It builds the whole tree (sees the checkmate), then looks back and says, "Wait, this specific branch over here actually led nowhere; let's cut it."

# Mathematics of DT

- A fully grown tree minimizes training impurity:

$$T_{full} = \arg \min_T R(T)$$

- Where:

$$R(T) = \sum_{m=1}^{|T|} N_m H_m$$

- $m$  = leaf index
  - $N_m$  = number of samples in leaf
  - $H_m$  = impurity (entropy or gini)
- A deep tree can make each leaf pure:
  - $H_m = 0$
  - So:  $R(T_{full}) = 0$
- But that means:
  - zero training error
  - extremely high variance
  - memorization
- So minimizing  $R(T)$  alone is dangerous

# Post-Pruning

- Instead of minimizing over all trees  $\operatorname{argmin}_T R(T)$  :
- We restrict hypothesis space:

$$\operatorname{argmin}_{T \in H_{\text{restricted}}} R(T)$$

# Post-Pruning Mathematically

- Now we introduce a proper regularized objective (Cost-Complexity Function  $R_\alpha(T)$ ).

- Define:

$$R_\alpha(T) = R(T) + \alpha |T|$$

- Where:

- $R(T)$  = empirical loss
- $|T|$  = number of leaves
- $\alpha$  = regularization strength

- This is directly analogous to (ridge regression):

$$\text{MSE} + \lambda \|w\|^2$$

- But instead of penalizing weights, we penalize **number of leaves**.

# Which one to use?

- In modern practice (like using XGBoost or LightGBM), we often use a bit of both.
- We use pre-pruning (max depth) to keep the model from exploding in size, and then use post-pruning (regularization/gamma) to fine-tune the complexity.